

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____

Преподаватель С.В. Умбетов

«22» сентября 2026 г.

Отчет по лабораторной работе №1
по дисциплине «Инженерно-техническая поддержка
сопровождения интеллектуальных информационных систем»
«Основы работы с GIT»

ЛР 09.02.08.19.002

Студент группы	<u>1ИИС-41</u>	<u>Н.Ю. Потапов</u>
	группа	инициалы., фамилия
Преподаватель	<u>старший преподаватель, доцент</u>	<u>С.В. Умбетов</u>
	должность, ученая степень	инициалы., фамилия

Барнаул 2026

Цель работы: освоить базовые и продвинутые приёмы работы с распределённой системой контроля версий Git в операционной системе Linux, изучить процесс установки и настройки Git, освоить основные команды для работы с локальным репозиторием, научиться управлять состояниями файлов, работать с ветками, выполнять слияния.

Задачи к работе: выполнить задания 1-20 и сделать вывод по работе:

- проверить наличие Git в системе и при необходимости установить его;
- настроить пользователя Git (имя и email);
- инициализировать локальный репозиторий;
- освоить команды работы с состояниями файлов (add, status, commit, diff);
- создать файл .gitignore и настроить игнорирование файлов;
- создать ветку, внести изменения, выполнить слияние и проанализировать историю.

Задание принял:



Подпись

Потапов Н.Ю.

ФИО

Ход работы:

Задание 1. Проверить наличие Git в системе.

Ход выполнения: открыл терминал и выполнил команду `git --version`. Система выдала ошибку «`bash: git: команда не найдена`», что свидетельствует о том, что Git в системе не установлен.

```
potapov_1iis_41@server:~$ git --version
-bash: git: команда не найдена
potapov_1iis_41@server:~$ _
```

Рисунок 1. Проверка наличия Git в системе — команда не найдена

Задание 2. Определить дистрибутив Linux.

Ход выполнения: выполнил команду `cat /etc/os-release`. В выводе увидел, что установлен Debian GNU/Linux 13 (trixie). Эта информация нужна для выбора правильного менеджера пакетов при установке Git.

```
potapov_1iis_41@server:~$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
DEBIAN_VERSION_FULL=13.6
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
potapov_1iis_41@server:~$ _
```

Рисунок 2. Информация о дистрибутиве Debian 13

Задание 3. Установить Git.

Ход выполнения: выполнил команду `sudo apt-get install git -y`. Менеджер пакетов apt скачал и установил Git версии 2.47.3 вместе с зависимостями.

```
potapov_1iis_41@server:~$ sudo apt-get install git -y
[sudo] пароль для potapov_1iis_41:
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Предлагаемые пакеты:
  git-doc git-email git-gui gitk gitweb git-cvs git-mediawiki git-svn
Следующие НОВЫЕ пакеты будут установлены:
  git
Обновлено 0 пакетов, установлено 1 новых пакетов, для удаления отмечено 0 пакетов, и 36 пакетов не обновлено.
Необходимо скачать 0 В/8 862 кВ архивов.
После данной операции объем занятого дискового пространства возрастёт на 50,3 МВ.
Выбор ранее не выбранного пакета git.
(Чтение базы данных ... на данный момент установлен 41951 файл и каталог.)
Подготовка к распаковке .../git_1%3a2.47.3-0+deb13u1_amd64.deb ...
Распаковывается git (1:2.47.3-0+deb13u1) ...
Настраивается пакет git (1:2.47.3-0+deb13u1) ...
potapov_1iis_41@server:~$
```

Рисунок 3. Процесс установки Git через apt-get

Задание 4. Проверить успешную установку Git

Ход выполнения: повторно выполнил команду `git --version`. В ответе получила строку «`git version 2.47.3`», что подтверждает успешную установку.

```
potapov_1iis_41@server:~$ git --version  
git version 2.47.3
```

Рисунок 4. Проверка версии установленного Git

Задание 5. Настроить имя пользователя Git.

Ход выполнения: выполнил команду `git config --global user.name "potapov_1iis_41"`. Теперь это имя будет указываться как автор во всех коммитах.

```
potapov_1iis_41@server:~$ git config --global user.name "potapov_1iis_41"
```

Рисунок 5. Настройка глобального имени пользователя

Задание 6. Настроить email пользователя Git.

Ход выполнения: выполнил команду `git config --global user.email "nikita.p.1iis41@gmail.com"`. Email будет записываться в метаданные каждого коммита.

```
potapov_1iis_41@server:~$ git config --global user.email nikita.p.1iis41@gmail.com
```

Рисунок 6. Настройка глобального email пользователя

Задание 7. Проверить применённые настройки.

Ход выполнения: выполнил команду `git config --list`. В выводе увидел установленные значения `user.name` и `user.email`, что подтверждает корректное применение настроек.

```
potapov_1iis_41@server:~$ git config --list  
user.name=potapov_1iis_41  
user.email=nikita.p.1iis41@gmail.com  
potapov_1iis_41@server:~$ _
```

Рисунок 7. Просмотр списка глобальных настроек Git

Задание 8. Создать рабочую директорию.

Ход выполнения: выполнил команду `mkdir git-practice` для создания папки и `cd git-practice` для перехода в неё.

```
potapov_1iis_41@server:~$ mkdir git-practice
potapov_1iis_41@server:~$ cd git-practice
potapov_1iis_41@server:~/git-practice$ _
```

Рисунок 8. Создание и переход в рабочую директорию

Задание 9. Инициализировать локальный репозиторий.

Ход выполнения: выполнил команду `git init`. Система создала пустой репозиторий Git в директории `/home/potapov_1iis_41/git-practice/.git` и выдала подсказку о том, что по умолчанию используется ветка `master`.

```
potapov_1iis_41@server:~/git-practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Инициализирован пустой репозиторий Git в /home/potapov_1iis_41/git-practice/.git/
```

Рисунок 9. Инициализация нового Git-репозитория

Задание 10. Убедиться в создании скрытой папки `.git`.

Ход выполнения: выполнил команду `ls -la`. В выводе увидел скрытую папку `.git`, в которой хранится вся информация о репозитории.

```
potapov_1iis_41@server:~/git-practice$ ls -la
итого 12
drwxrwxr-x 3 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:16 .
drwxr-xr-x 6 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:13 ..
drwxrwxr-x 7 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:16 .git
potapov_1iis_41@server:~/git-practice$
```

Рисунок 10. Проверка наличия папки `.git` через `ls -la`

Задание 11. Создать тестовые файлы.

Ход выполнения: командами `touch readme.txt`, `touch script.sh` и `touch data.json` создал три пустых файла. Командой `ls -la` убедился, что файлы появились в директории.

```

potapov_1iis_41@server:~/git-practice$ touch readme.txt
potapov_1iis_41@server:~/git-practice$ touch script.sh
potapov_1iis_41@server:~/git-practice$ touch data.json
potapov_1iis_41@server:~/git-practice$ ls -la
итого 12
drwxrwxr-x 3 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:20 .
drwxr-xr-x 6 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:13 ..
-rw-rw-r-- 1 potapov_1iis_41 potapov_1iis_41  0 сен 22 16:20 data.json
drwxrwxr-x 7 potapov_1iis_41 potapov_1iis_41 4096 сен 22 16:16 .git
-rw-rw-r-- 1 potapov_1iis_41 potapov_1iis_41  0 сен 22 16:20 readme.txt
-rw-rw-r-- 1 potapov_1iis_41 potapov_1iis_41  0 сен 22 16:20 script.sh
potapov_1iis_41@server:~/git-practice$

```

Рисунок 11. Создание файлов *readme.txt*, *script.sh* и *data.json*

Задание 12. Проверить статус репозитория.

Ход выполнения: выполнил команду `git status`. Система показала, что все три созданных файла находятся в состоянии *untracked* (неотслеживаемые), так как они ещё не добавлены в репозиторий.

```

potapov_1iis_41@server:~/git-practice$ git status
Текущая ветка: master

Еще нет коммитов

Неотслеживаемые файлы:
(используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
    data.json
    readme.txt
    script.sh

индекс пуст, но есть неотслеживаемые файлы
(используйте «git add», чтобы проиндексировать их)
potapov_1iis_41@server:~/git-practice$ _

```

Рисунок 12. Статус репозитория — файлы в состоянии *untracked*

Задание 13. Добавить в индекс только текстовые файлы.

Ход выполнения: выполнил команду `git add *.txt` с использованием регулярного выражения. Командой `git status` проверил, что в состояние *staged* попал только файл *readme.txt*, а остальные остались *untracked*.

```

potapov_1iis_41@server:~/git-practice$ git add *.txt
potapov_1iis_41@server:~/git-practice$ git status
Текущая ветка: master

Еще нет коммитов

Изменения, которые будут включены в коммит:
(используйте «git rm --cached <файл>...», чтобы убрать из индекса)
    новый файл:   readme.txt

Неотслеживаемые файлы:
(используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
    data.json
    script.sh

potapov_1iis_41@server:~/git-practice$

```

Рисунок 13. Добавление в индекс только файлов с расширением *.txt*

Задание 14. Добавить все остальные файлы.

Ход выполнения: выполнил команду `git add -A`, которая добавила в индекс все оставшиеся файлы. Повторная проверка `git status` показала, что все три файла (`data.json`, `readme.txt`, `script.sh`) теперь в состоянии `staged`.

```
potapov_1iis_41@server:~/git-practice$ git add -A
potapov_1iis_41@server:~/git-practice$ git status
Текущая ветка: master

Еще нет коммитов

Изменения, которые будут включены в коммит:
  (используйте «git rm --cached <файл>...», чтобы убрать из индекса)
    новый файл:   data.json
    новый файл:   readme.txt
    новый файл:   script.sh

potapov_1iis_41@server:~/git-practice$
```

Рисунок 14. Добавление всех файлов в индекс командой `git add -A`

Задание 15. Создать первый коммит.

Ход выполнения: выполнил команду `git commit -m "initial commit: add base files"`. Git создал коммит с хешем `77f2c54`, в который вошли все три файла.

```
potapov_1iis_41@server:~/git-practice$ git commit -m "initial commit: add base files"
[master (корневой коммит) 77f2c54] initial commit: add base files
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 data.json
create mode 100644 readme.txt
create mode 100644 script.sh
potapov_1iis_41@server:~/git-practice$
```

Рисунок 15. Создание первого коммита с сообщением

Задание 16. Просмотреть историю коммитов.

Ход выполнения: выполнил команду `git log`. В выводе увидел метаданные коммита: хеш `77f2c54`, автора `potapov_1iis_41`, дату `Tue Sep 22 16:29:14 2026` и сообщение «`initial commit: add base files`».

```
potapov_1iis_41@server:~/git-practice$ git log
commit 77f2c54b3f2dc4b94418b431f4426f7b0a3d44c0 (HEAD -> master)
Author: potapov_1iis_41 <nikita.p.1iis41@gmail.com>
Date:   Tue Sep 22 16:29:14 2026 +0700

    initial commit: add base files
potapov_1iis_41@server:~/git-practice$
```

Рисунок 16. История коммитов с метаданными

Задание 17. Внести изменения и просмотреть различия.

Ход выполнения: открыл файл `readme.txt` в редакторе и добавил строку «i hate python». Выполнил команду `git diff`, которая показала постранные различия между индексом и рабочей директорией.

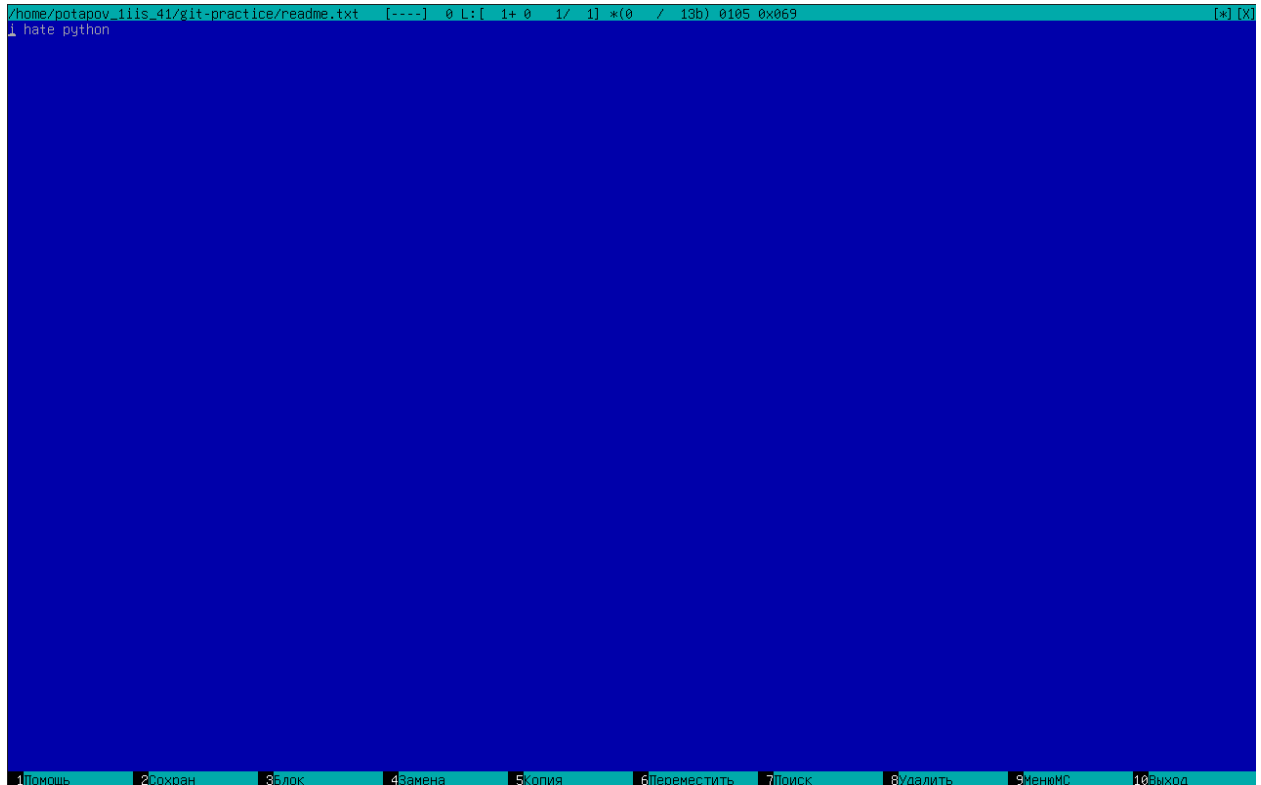


Рисунок 17. Изменение файла `readme.txt` в МС

```
potapov_1iis_41@server:~/git-practice$ git diff
diff --git a/readme.txt b/readme.txt
index e69de29..f616a05 100644
--- a/readme.txt
+++ b/readme.txt
@@ -0,0 +1 @@
+i hate python
\ No newline at end of file
potapov_1iis_41@server:~/git-practice$
```

Рисунок 18. Просмотр различий командой `git diff`

Задание 18. Создать файл .gitignore.

Ход выполнения: выполнил команду `touch .gitignore` и через редактор добавил в него шаблоны `*.log`, `temp/`, `*.tmp`, `.cache/` для игнорирования временных и кэш-файлов. Командой `git status` проверил, что сам файл `.gitignore` отображается как `untracked`.

```
potapov_1iis_41@server:~/git-practice$ touch .gitignore
potapov_1iis_41@server:~/git-practice$ _
```

Рисунок 19. Создание файла .gitignore

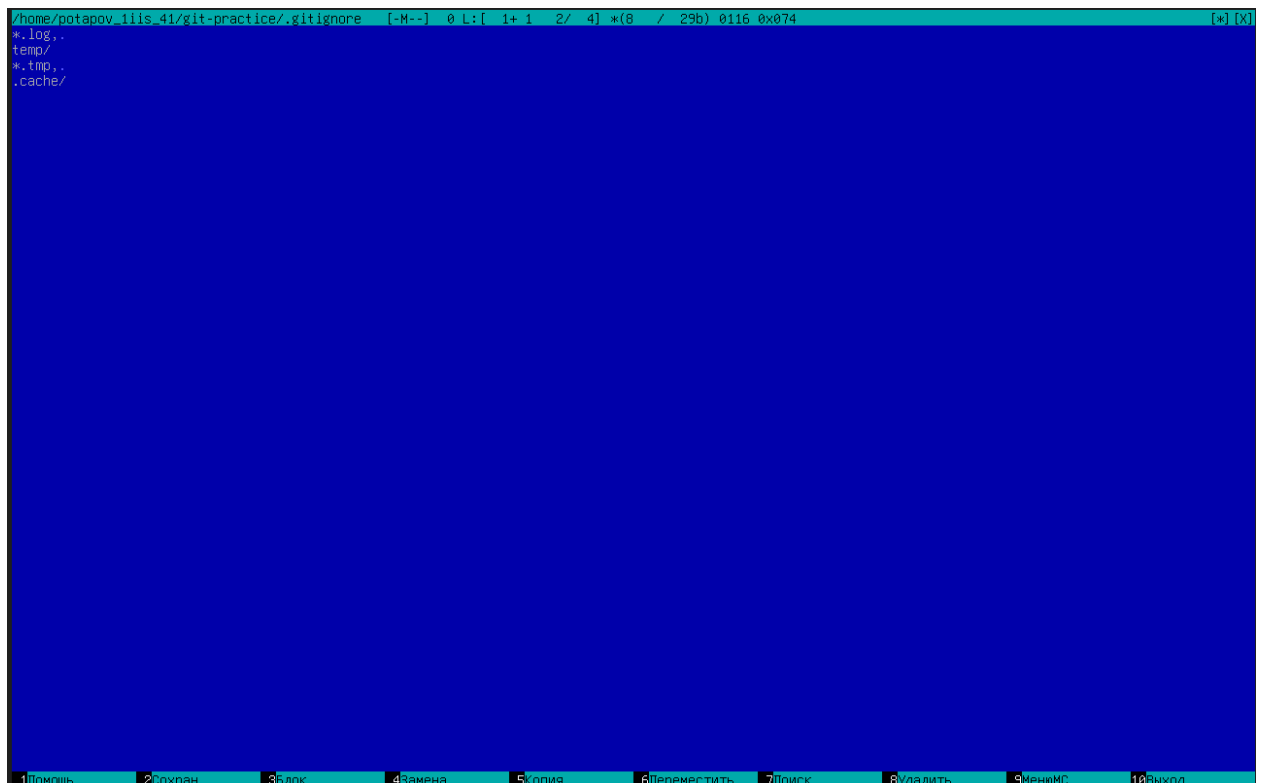


Рисунок 20. Изменение файла .gitignore в МС

```
potapov_1iis_41@server:~/git-practice$ git status
Текущая ветка: master
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (используйте «git restore <файл>...», чтобы отменить изменения в рабочем каталоге)
      изменено:   readme.txt
Неотслеживаемые файлы:
  (используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
      .gitignore
индекс пуст (используйте «git add» и/или «git commit -a»)
```

Рисунок 21. Проверка что файл .gitignore неотслеживается Git

Задание 19. Создать новую ветку.

Ход выполнения: выполнил команду `git checkout -b feature/update-readme`. Система создала новую ветку на основе текущей и переключилась на неё. Командой `git branch` убедился, что теперь активна ветка `feature/update-readme`.

```
potapov_1iis_41@server:~/git-practice$ git checkout -b feature/update-readme
Переключились на новую ветку «feature/update-readme»
potapov_1iis_41@server:~/git-practice$ git branch
* feature/update-readme
  master
potapov_1iis_41@server:~/git-practice$ _
```

Рисунок 22. Создание ветки `feature/update-readme` и переключение на неё

```
potapov_1iis_41@server:~/git-practice$ git status
Текущая ветка: feature/update-readme
```

Рисунок 23. Проверка выбранной ветки

Задание 20. Внести изменения, закоммитить и выполнить слияние.

Ход выполнения: в ветке `feature/update-readme` дополнил файл `readme.txt` строкой «i love nier automata and buy it», затем выполнил `git commit -m "commit 2: redact readme.txt"`. Переключился на `master` командой `git checkout master` и выполнил слияние `git merge feature/update-readme`. Слияние прошло по схеме Fast-forward. Командой `git log --oneline --graph --all` просмотрел итоговую историю в виде графа.

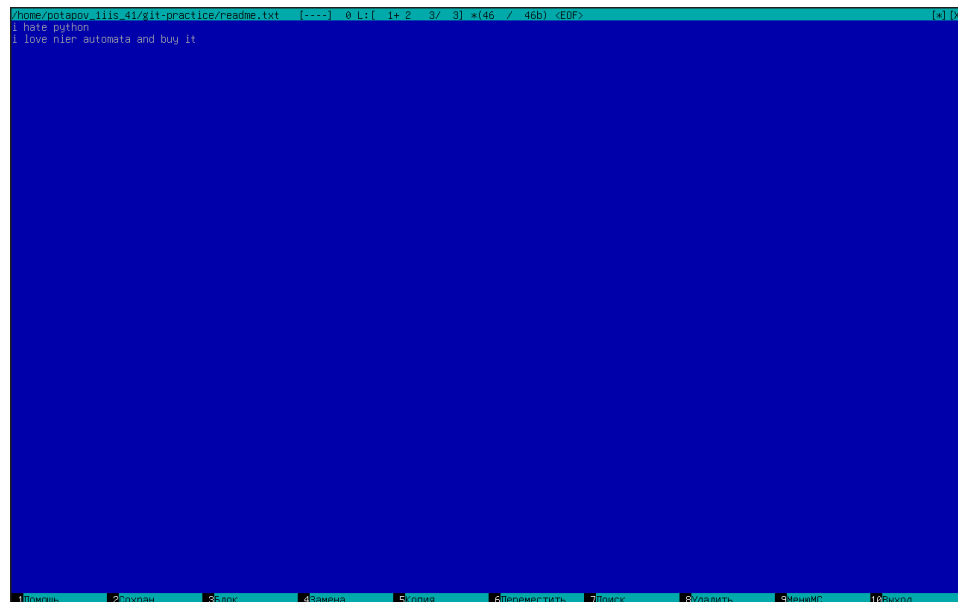


Рисунок 24. Изменение файла `readme.txt` в МС

```

potapov_1iis_41@server:~/git-practice$ git commit -m "commit 2: redact readme.txt"
Текущая ветка: feature/update-readme
Изменения, которые не в индексе для коммита:
  (используйте «git add <файл>...», чтобы добавить файл в индекс)
  (используйте «git restore <файл>...», чтобы отменить изменения в рабочем каталоге)
      изменено:      readme.txt

Неотслеживаемые файлы:
  (используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
      .gitignore

индекс пуст (используйте «git add» и/или «git commit -a»)
potapov_1iis_41@server:~/git-practice$ _

```

Рисунок 25. Создание коммита в новой ветке с комментарием

```

potapov_1iis_41@server:~/git-practice$ git checkout master
M      readme.txt
Переключились на ветку «master»
potapov_1iis_41@server:~/git-practice$ _

```

Рисунок 26. Переключение на ветку «master»

```

potapov_1iis_41@server:~/git-practice$ git merge feature/update-readme
Обновление 77f2c54..be3fd74
Fast-forward
  readme.txt | 2 ++
  1 file changed, 2 insertions(+)
potapov_1iis_41@server:~/git-practice$ _

```

Рисунок 27. Объединение двух веток

```

potapov_1iis_41@server:~/git-practice$ git log --oneline --graph --all
* be3fd74 (HEAD -> master, feature/update-readme) commit 2: redact readme.txt
* 77f2c54 initial commit: add base files
potapov_1iis_41@server:~/git-practice$ _

```

Рисунок 28. Итоговая история коммитов после слияния веток

Вывод:

В ходе лабораторной работы я освоил практические навыки работы с системой контроля версий Git в ОС Linux. На практике выполнил: установку и глобальную настройку Git, инициализацию локального репозитория, управление состояниями файлов (untracked, staged, committed) с помощью команд add и commit. Изучил инструменты анализа изменений (git diff, git log), создание файла .gitignore для игнорирования файлов по шаблону, а также полный цикл работы с ветками: создание ветки, фиксация изменений, переключение и слияние (git merge) с визуализацией итоговой истории коммитов. Приобретённые навыки позволяют мне эффективно использовать Git для версионного контроля и командной разработки.

Ответы на контрольные вопросы:

1. Что такое Git и для чего он используется?

Распределенная система контроля версий. Нужна для отслеживания изменений в коде, быстрого отката к предыдущим версиям и безопасной совместной работы.

2. В чем разница между локальным и удаленным репозиторием?

Локальный хранит всю историю на вашем компьютере в папке `.git`. Удаленный находится на сервере, а у вас хранится только его копия для синхронизации через `push` и `pull`.

3. Какая команда проверяет наличие Git и его версию?

`git --version`

4. Команды установки Git в разных дистрибутивах:

Ubuntu/Debian: `sudo apt-get install git`

Fedora: `sudo dnf install git`

Arch Linux: `sudo pacman -S git`

5. Для чего нужна настройка `user.name` и `user.email`?

Для идентификации автора. Эти данные автоматически записываются в метаданные каждого созданного коммита.

6. Разница между `git config --global` и `git config` без флага?

`--global` применяет настройки ко всем репозиториям пользователя на компьютере. Без флага настройки применяются только к текущему репозиторию.

7. Что создается при выполнении `git init`?

Скрытая папка `.git`, которая содержит всю базу данных, историю коммитов и настройки репозитория.

8. Для чего папка `.git` и можно ли ее удалять?

Она хранит всю историю версий. Удалять нельзя, это уничтожит историю коммитов и превратит проект в обычную папку.

9. Четыре состояния файлов в Git:

Не отслеживаемое (untracked), измененное (modified), подготовленное (staged), закоммиченное (committed).

10. Разница между untracked и modified?

Untracked - новый файл, Git о нем еще не знает. Modified - файл уже под контролем Git, но в него внесли изменения, которые пока не добавлены в индекс.

11. Отличие git add ., git add -A и git add <имя файла>?

<имя файла> добавляет только этот конкретный файл.

. добавляет новые и измененные файлы из текущей папки и вложенных.

-A добавляет абсолютно все изменения, включая удаленные файлы, во всем проекте.

12. Что такое коммит и его метаданные?

Фиксированный снимок состояния проекта. Содержит уникальный хеш, имя автора, дату и сообщение об изменениях.

13. Для чего нужна команда git status?

Показывает текущее состояние репозитория: активную ветку, а также списки измененных, подготовленных и не отслеживаемых файлов.

14. Что показывает git diff и когда применяется?

Показывает строчные различия между файлами в рабочей директории и в индексе. Применяется перед коммитом для проверки того, что именно изменилось.

15. Для чего файл .gitignore и как пишутся шаблоны?

Заставляет Git игнорировать ненужные файлы, например логи или кэш. Шаблоны пишутся строчно: конкретные имена, маски расширений или папки.

16. Что такое ветка и почему это подвижный указатель?

Это указатель на конкретный коммит. Она подвижная, потому что при создании нового коммита указатель автоматически сдвигается на него.

17. Разница между `git checkout master` и `git checkout -b new-branch`?

`checkout master` просто переключает на уже существующую ветку. `checkout -b new-branch` создает новую ветку и сразу переключает на нее.

18. Что делает `git merge` и когда возникает конфликт?

Объединяет изменения из указанной ветки в текущую. Конфликт возникает, если в обеих ветках были изменены одни и те же строки одного файла.

19. Как разрешить конфликт и какие маркеры используются?

Вручную редактируем файл, удаляя маркеры:

<<<<<<< HEAD (ваши изменения),

=====
(разделитель),

branch-name (изменения из сливаемой ветки).

После правки делаем `git add` и `git commit`.

20. Для чего `git log` и что делают флаги `--oneline` `--graph` `--all`?

Показывает историю коммитов.

`--oneline` выводит каждый коммит кратко в одну строку.

`--graph` рисует текстовую схему ветвления.

`--all` показывает историю по всем веткам репозитория.